

HACKATHON DAY 2

Date

Marketplace Technical Foundation

General E-Commerce Marketplace (Clothing)

System Architecture Document:

Overview:

The system architecture for the clothing E-commerce marketplace connects a dynamic frontend with a solid backend managed through Sanity CMS. Third-party APIs are used for shipment tracking and secure payment processing. This modular design ensures easy scaling and smooth user experiences.

Components:

1. Frontend (Next.js):

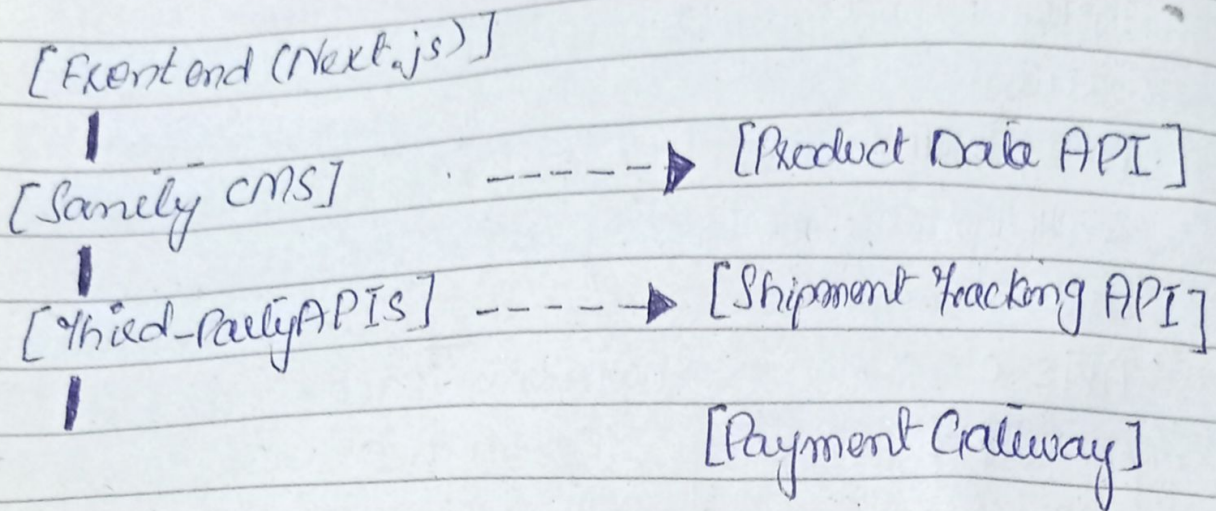
- Dynamic, responsive pages for browsing clothes, managing cart items, and checkout.
- Communicates with the backend APIs for real-time data updates.

2. Sanity CMS:

- Manages product details, customer profiles, and order data.
- Support flexible, custom schemas for easy data management.

3. Third-Party APIs:

- Shipment Tracking API: Provides real-time updates on delivery status.
- Payment Gateway API: Handles secure transactions for payments.

Architecture Diagram:Data Flow:

1. Users browse products and make orders on the frontend.
2. The frontend sends data requests to the Family CMS via API.
3. Orders are recorded in Family CMS, with updates fetched from third-party APIs.
4. Shipment and payment updates are displayed to users in real-time.

API Specification Document:API Endpoints:

Endpoint	Method	Purpose	Example Payload/Response
/products	GET	Fetch all clothing product details.	Response: { "id": 1, "name": "T-shirt", "price": 20 }
/orders	POST	Create a new clothing order	Payload: { "customerId": 123, "products": [...] } Response: { "id": 1, "status": "pending" }

/shipment	GET	Track clothing order shipment status	Response: { "shipmentId": 456, "status": "Delivered" }
/loyalty-points	GET	Fetch loyalty points for a customer	Response: { "customerId": 123, "points": 500 }

Endpoint Example Endpoint Details:

1. /products:

- Method: GET.
- Description: Fetches all available clothing products.
- Response: { "id": 1, "name": "T-shirt", "price": 20, "stock": 50 }

2. /orders:

- Method: POST.
- Description: Create a new clothing order in the system.
- Payload: { "customerId": 123, "products": [{ "productId": 1, "quantity": 2 }] }
- Response: { "orderId": 789, "status": "Pending" }

Workflow Diagram:

Workflows:

1. User Registration:

- User signs up → Data saved in Sanity CMS → confirmation email sent

2. Product Browsing:

- User searches for clothing → Sanity CMS sends product data → Results displayed dynamically.

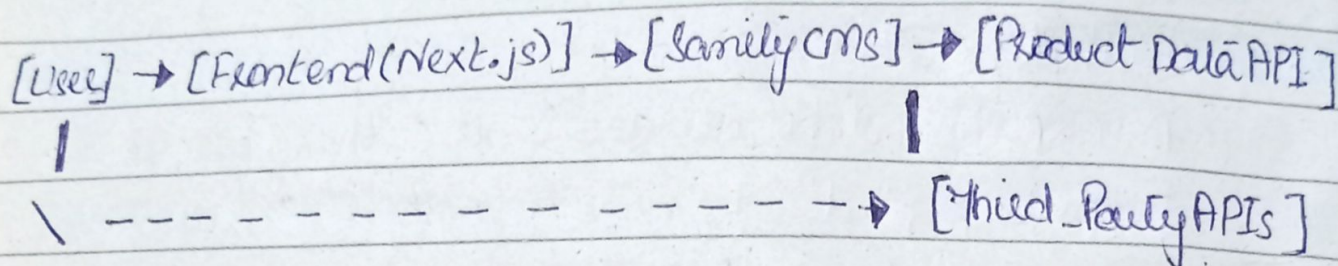
3. Order Placement:

- User adds items to cart → Proceed to checkout →
- Order saved in Sanity CMS → Payment processed.

4. Shipment Tracking:

- Shipment API gets status → Updates displayed on user dashboard.

Diagram:



A visual representation will show how users interact with the system, from product browsing to order completion and tracking.

Data Schema Design:

Entities and Attributes:

1. Products:

- Product ID.
- Name.
- Description.
- Price.
- Stock Level.
- Category (e.g., Men, Women, Kids).
- Sub-Category (e.g., Casual, Formal).
- Images.
- Tags (e.g., "New Arrival", "Sale").

2. Order:

- Order ID.
- Customer ID.
- Product List (Product ID, Quantity)
- Total Amount.
- Status (Pending, Shipped, Delivered)
- Payment Method.
- Time Stamps (Order Date, Expected Delivery Date)

3. Customers:

- Customer ID.
- Name.
- Email.
- Address.
- Contact Number.
- Order History.
- Preferences (e.g., Size, Color Preferences).

4. Shipment:

- Shipment ID.
- Order ID.
- Delivery Address.
- Delivery Status.
- Carrier Partner.
- Expected Delivery Date.

5. Delivery Zones:

- Zone ID.
- Area Name.
- Coverage (Cities / Regions)

Co. Loyalty Points:

- Points ID
- Customer ID
- Points Earned
- Points Redeemed
- Expiry Date

Technical Roadmap:Phase 1: Initial Setup

1. Define business goals and target audience.
2. Draft data schema based on the clothing marketplace needs.

3. Set up Sanity CMS for backend management.

Phase 2: System Architecture and API Development

1. Design system architecture and workflows.
2. Develop API endpoints for products, orders, and shipment tracking.
3. Integrate third-party APIs for shipment and payment processing.

Phase 3: Frontend Development

1. Build responsive pages with Next.js.
2. Implement real-time data fetching from APIs.
3. Develop dynamic features like product search and filtering.

Phase 4: Testing and Deployment

1. Conduct unit and integration tests for APIs and frontend components.

2. Deploy the application on a scalable platform.
3. Monitor performance and resolve any issues.

Phase 5: Post-Launch Optimization

1. Collect user feedback and make iterative improvements.
2. Introduce new features such as seasonal collections and loyalty programs.
3. Scale infrastructure to handle higher traffic and orders.